INFOSYS SPRINGBOARD 6.0

Name- Swapnil Pal
Topic- Smart Energy Consumption Analysis and Prediction using Machine Learning with Device-Level Insights

Documentation for Week 1-2

Smart Home Energy Consumption

Data Preprocessing, Visualization & Machine Learning Documentation

1. Project Objective

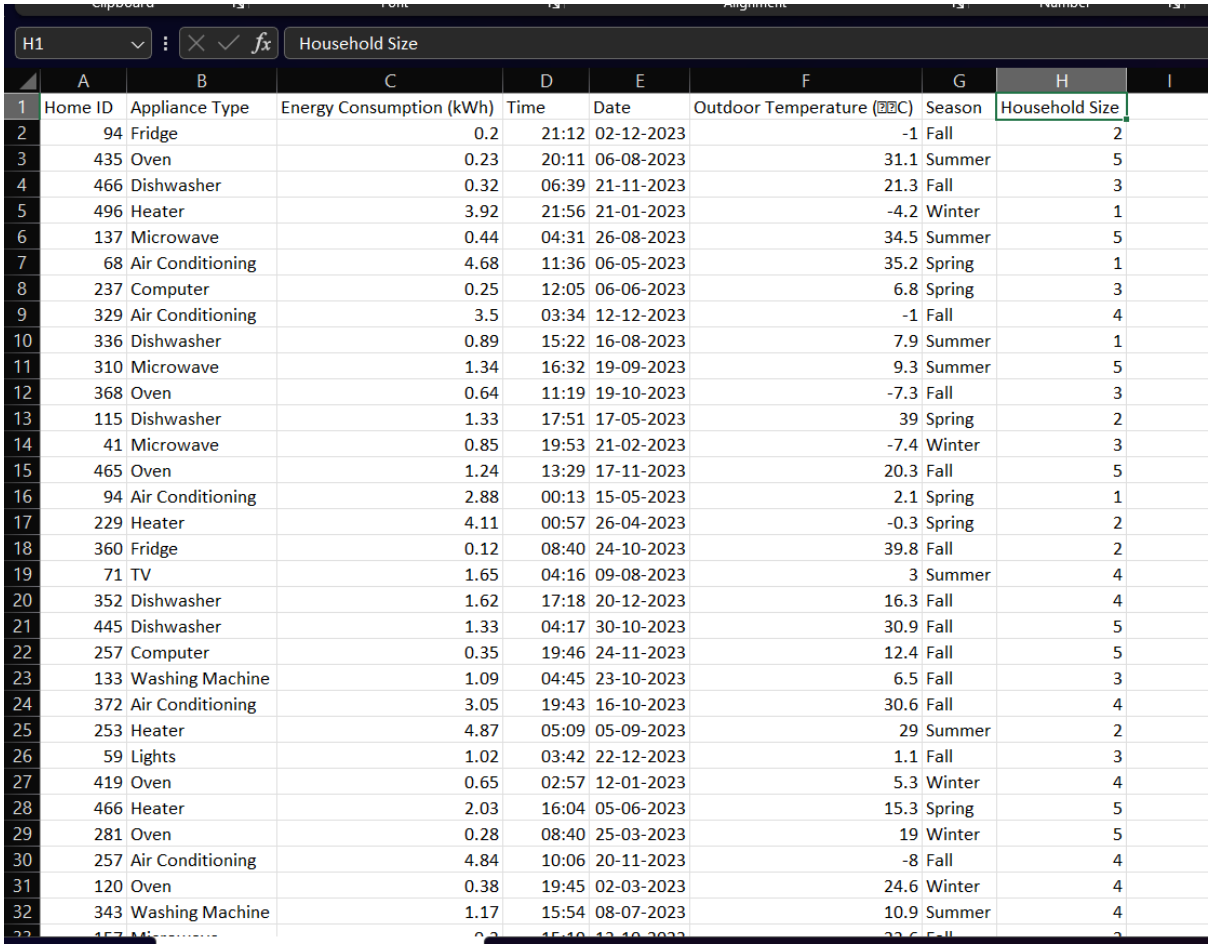
The objective of this project is to **analyse and predict smart home energy consumption** using historical data. The project focuses on understanding how factors such as appliance type, time, season, temperature, and household size affect energy usage, and to prepare the data for machine learning models.

2. Dataset Description

The dataset contains smart home energy usage records with the following attributes:

- Home ID
- Appliance Type
- Energy Consumption (kWh)
- Time
- Date
- Outdoor Temperature (°C)
- Season
- Household Size

The dataset consists of **100,000 records** and is suitable for exploratory data analysis and machine learning.



H1 | Household Size

	A	B	C	D	E	F	G	H	I
1	Home ID	Appliance Type	Energy Consumption (kWh)	Time	Date	Outdoor Temperature (□□C)	Season	Household Size	
2	94	Fridge	0.2	21:12	02-12-2023	-1	Fall	2	
3	435	Oven	0.23	20:11	06-08-2023	31.1	Summer	5	
4	466	Dishwasher	0.32	06:39	21-11-2023	21.3	Fall	3	
5	496	Heater	3.92	21:56	21-01-2023	-4.2	Winter	1	
6	137	Microwave	0.44	04:31	26-08-2023	34.5	Summer	5	
7	68	Air Conditioning	4.68	11:36	06-05-2023	35.2	Spring	1	
8	237	Computer	0.25	12:05	06-06-2023	6.8	Spring	3	
9	329	Air Conditioning	3.5	03:34	12-12-2023	-1	Fall	4	
10	336	Dishwasher	0.89	15:22	16-08-2023	7.9	Summer	1	
11	310	Microwave	1.34	16:32	19-09-2023	9.3	Summer	5	
12	368	Oven	0.64	11:19	19-10-2023	-7.3	Fall	3	
13	115	Dishwasher	1.33	17:51	17-05-2023	39	Spring	2	
14	41	Microwave	0.85	19:53	21-02-2023	-7.4	Winter	3	
15	465	Oven	1.24	13:29	17-11-2023	20.3	Fall	5	
16	94	Air Conditioning	2.88	00:13	15-05-2023	2.1	Spring	1	
17	229	Heater	4.11	00:57	26-04-2023	-0.3	Spring	2	
18	360	Fridge	0.12	08:40	24-10-2023	39.8	Fall	2	
19	71	TV	1.65	04:16	09-08-2023	3	Summer	4	
20	352	Dishwasher	1.62	17:18	20-12-2023	16.3	Fall	4	
21	445	Dishwasher	1.33	04:17	30-10-2023	30.9	Fall	5	
22	257	Computer	0.35	19:46	24-11-2023	12.4	Fall	5	
23	133	Washing Machine	1.09	04:45	23-10-2023	6.5	Fall	3	
24	372	Air Conditioning	3.05	19:43	16-10-2023	30.6	Fall	4	
25	253	Heater	4.87	05:09	05-09-2023	29	Summer	2	
26	59	Lights	1.02	03:42	22-12-2023	1.1	Fall	3	
27	419	Oven	0.65	02:57	12-01-2023	5.3	Winter	4	
28	466	Heater	2.03	16:04	05-06-2023	15.3	Spring	5	
29	281	Oven	0.28	08:40	25-03-2023	19	Winter	5	
30	257	Air Conditioning	4.84	10:06	20-11-2023	-8	Fall	4	
31	120	Oven	0.38	19:45	02-03-2023	24.6	Winter	4	
32	343	Washing Machine	1.17	15:54	08-07-2023	10.9	Summer	4	

3. Data Loading and Initial Inspection

The dataset is loaded using Pandas. Initial inspection is performed using:

- head() to view sample rows
- shape to check dataset size
- columns to identify features
- describe() for basic statistics

This step helps understand the structure and scale of the data.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
df = pd.read_csv('/content/smart_home_energy_consumption_large.csv')
display(df.head())
```

	Home ID	Appliance Type	Energy Consumption (kWh)	Time	Date	Outdoor Temperature (°C)	Season	Household Size
0	94	Fridge	0.20	21:12	2023-12-02	-1.0	Fall	2
1	435	Oven	0.23	20:11	2023-08-06	31.1	Summer	5
2	466	Dishwasher	0.32	06:39	2023-11-21	21.3	Fall	3
3	496	Heater	3.92	21:56	2023-01-21	-4.2	Winter	1
4	137	Microwave	0.44	04:31	2023-08-26	34.5	Summer	5

```
df.describe()
```

	Home ID	Energy Consumption (kWh)	Outdoor Temperature (°C)	Household Size
count	100000.000000	100000.000000	100000.000000	100000.000000
mean	250.374980	1.499952	14.950135	3.001770
std	144.435367	1.181176	14.438755	1.417077
min	1.000000	0.100000	-10.000000	1.000000
25%	125.000000	0.590000	2.400000	2.000000
50%	250.000000	1.230000	14.900000	3.000000
75%	375.000000	1.870000	27.400000	4.000000
max	500.000000	5.000000	40.000000	5.000000

4. Date and Time Processing

4.1 Date Conversion

The Date column is converted into a datetime format to allow time-based analysis.

4.2 Hour Extraction

The Time column is converted into an Hour feature (0–23), as energy consumption strongly depends on the time of day.

Purpose:
To enable hourly energy usage analysis and improve feature quality for ML models.

```python
# Convert Date column
df['Date'] = pd.to_datetime(df['Date'])

# Convert Time to Hour
df['Hour'] = pd.to_datetime(df['Time'], format='%H:%M').dt.hour

print("Date & Hour conversion successful")
df[['Date', 'Time', 'Hour']].head()
```

```
Date & Hour conversion successful
```

	Date	Time	Hour
0	2023-12-02	21:12	21
1	2023-08-06	20:11	20
2	2023-11-21	06:39	6
3	2023-01-21	21:56	21
4	2023-08-26	04:31	4

5. Data Validation

Data types are verified to ensure correct conversions.
Missing values are checked to confirm data integrity.

Result:
No missing values were found, indicating a clean dataset.

6. Timestamp Creation

A combined Timestamp column is created using Date and Time.

Purpose:

- Enables chronological sorting
- Required for time-series features like lag and rolling averages

```
df['Timestamp'] = pd.to_datetime(
    df['Date'].astype(str) + ' ' + df['Time']
)

print("Timestamp column created")
df[['Date', 'Time', 'Timestamp']].head()
```

```
Timestamp column created
```

	Date	Time	Timestamp
0	2023-12-02	21:12	2023-12-02 21:12:00
1	2023-08-06	20:11	2023-08-06 20:11:00
2	2023-11-21	06:39	2023-11-21 06:39:00
3	2023-01-21	21:56	2023-01-21 21:56:00
4	2023-08-26	04:31	2023-08-26 04:31:00

7. Data Sorting

The dataset is sorted by Home ID and Timestamp.

8. Outlier Detection and Removal

Method Used: Interquartile Range (IQR)

Outliers are identified using:

- Q1 (25th percentile)
- Q3 (75th percentile)
- IQR = Q3 − Q1

Values outside Q1 − 1.5 × IQR and Q3 + 1.5 × IQR are removed.

Result:

- Outliers removed: ~1,900 records
- Data becomes more realistic and stable

Purpose:

- Prevents extreme values from misleading ML models

- Improves visualization clarity and prediction accuracy

```
[21]
✓ 1s
```

```python
before = df.shape[0]

Q1 = df['Energy Consumption (kWh)'].quantile(0.25)
Q3 = df['Energy Consumption (kWh)'].quantile(0.75)
IQR = Q3 - Q1

df = df[
    (df['Energy Consumption (kWh)'] >= Q1 - 1.5 * IQR) &
    (df['Energy Consumption (kWh)'] <= Q3 + 1.5 * IQR)
]

after = df.shape[0]


print("Outlier Removal Completed ")
print(f"Rows before outlier removal: {before}")
print(f"Rows after outlier removal:  {after}")
print(f"Outliers removed:           {before - after}")


plt.figure(figsize=(12, 6))

sns.boxplot(
    x='Appliance Type',
    y='Energy Consumption (kWh)',
    data=df
)

plt.xlabel('Appliance Type')
plt.ylabel('Energy Consumption (kWh)')
plt.title('Energy Consumption Distribution by Appliance Type')
plt.xticks(rotation=45)

plt.show()
```

```
Outlier Removal Completed
Rows before outlier removal: 92003
Rows after outlier removal:  90077
Outliers removed:           1926
```

9. Feature Scaling (Standardization)

Energy consumption values are scaled using **StandardScaler**.

What scaling does:

- Converts values to a standard scale

- Mean becomes ~0
- Standard deviation becomes ~1

Why scaling is needed:

- ML models perform better when features are on similar scales
- Prevents large values from dominating learning

Scaled values indicate how far a value is from the average in terms of standard deviation.

10. Time-Based Feature Engineering

Additional time features are extracted:

- Day
- Month
- Weekday

Purpose:
Energy consumption varies by:

- Day of month
- Season
- Weekdays vs weekends

These features help the model learn behavioural and seasonal patterns.

11. Exploratory Visualization (Boxplot)

A boxplot is used to visualize **energy consumption by appliance type**.

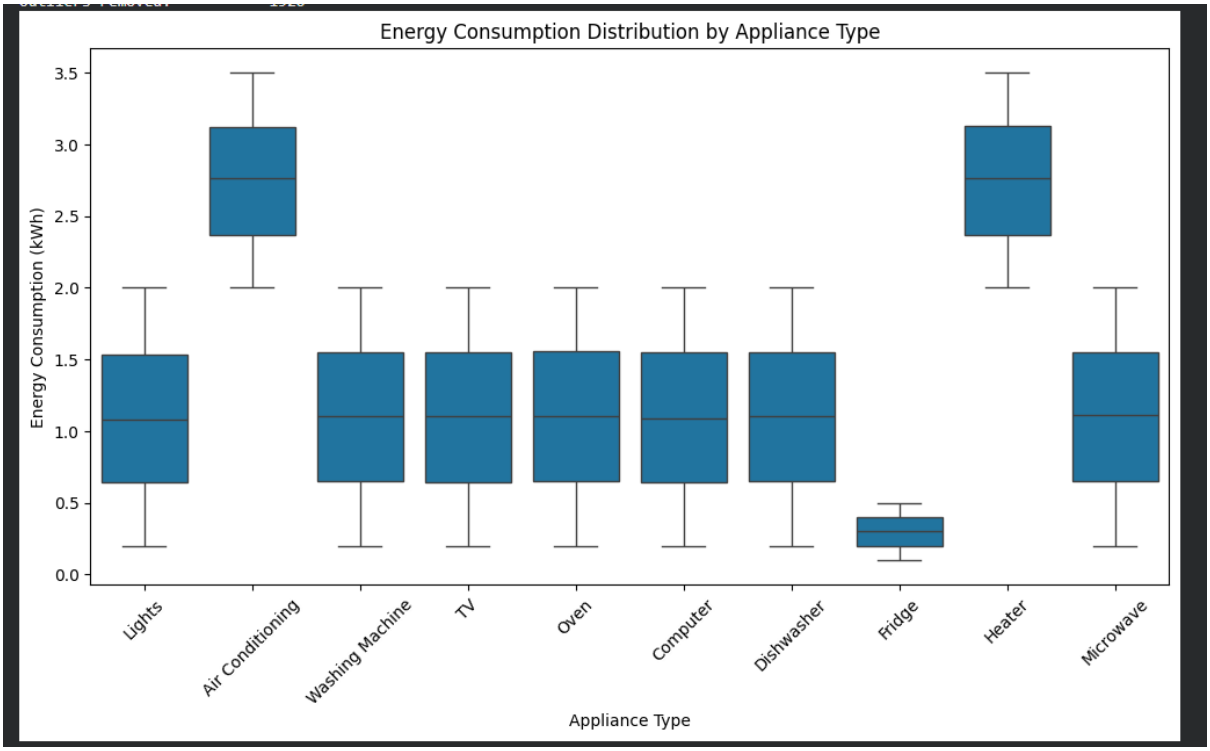
Insights:

- Heaters and air conditioners consume the most energy
- Refrigerators consume the least but consistently
- Boxplots clearly show variability and median usage

Purpose:

- Identify energy-intensive appliances

- Support smart energy optimization decisions



12. Categorical Encoding

Categorical features (Appliance Type, Season) are converted into numerical form using **one-hot encoding**.

Why encoding is required:

Machine learning models cannot process text values.

Why drop_first=True:

- Prevents dummy variable trap
- Reduces redundancy
- Improves model stability

13. Removing Redundant Columns

Columns such as Time and Timestamp are removed.

Reason:

- Information is already captured through engineered features
- Avoids redundancy and noise

14. Feature–Target Separation

The dataset is split into:

- **X (features)** – input variables
- **y (target)** – energy consumption

The target column is removed from X to prevent **data leakage**.

Purpose:
Ensures the model learns patterns instead of memorizing answers.

15. Train–Validation–Test Split

```
[5]
0s
    # First split: Train (70%) + Temp (30%)
    X_train, X_temp, y_train, y_temp = train_test_split(
        X, y, test_size=0.30, random_state=42
    )

    # Second split: Validation (15%) + Test (15%)
    X_val, X_test, y_val, y_test = train_test_split(
        X_temp, y_temp, test_size=0.50, random_state=42
    )

    print("Dataset split completed \n")

    print("Training set size:")
    print("X_train:", X_train.shape, "y_train:", y_train.shape)

    print("\nValidation set size:")
    print("X_val:", X_val.shape, "y_val:", y_val.shape)

    print("\nTest set size:")
    print("X_test:", X_test.shape, "y_test:", y_test.shape)
```

```
Dataset split completed

Training set size:
X_train: (64402, 21) y_train: (64402,)

Validation set size:
X_val: (13800, 21) y_val: (13800,)

Test set size:
X_test: (13801, 21) y_test: (13801,)
```

The dataset is split into:

- 70% Training
- 15% Validation

- 15% Testing

Why this split is used:

- Training set → learn patterns
- Validation set → tune model
- Test set → unbiased performance evaluation

A fixed random state ensures reproducibility.

16. Final Outcome

At the end of this pipeline:

- The dataset is clean
- Features are meaningful and numeric
- Data is ML-ready
- The project follows industry-standard practices

17. Conclusion

This systematic preprocessing and feature engineering approach ensures accurate, reliable, and unbiased prediction of smart home energy consumption. The prepared dataset can now be used effectively for machine learning models such as Linear Regression and Random Forest.